

Healthcare CTMS Automation Framework

1. Framework Overview

This framework is designed for enterprise Healthcare Clinical Trial Management System (CTMS) applications using:

- Playwright
- TypeScript
- Cucumber BDD
- Allure Reporting
- Azure DevOps CI/CD
- MFA Authentication Support
- Cross-browser execution

The framework supports:

- Scalable automation architecture
- Reusable test components
- Enterprise-level reporting
- Healthcare compliance readiness
- Maintainable automation standards

2. Complete Framework Folder Structure

```
Healthcare-CTMS-Automation/  
|  
├── config/  
|   ├── env/  
|   |   ├── qa.env  
|   |   ├── stage.env  
|   |   └── prod.env  
|   ├── playwright.config.ts  
|   ├── cucumber.js  
|   ├── globalSetup.ts  
|   └── globalTeardown.ts  
└── src/  
    ├── pages/  
    |   ├── BasePage.ts  
    |   └── LoginPage.ts
```

```
|
|
|   └─ DashboardPage.ts
|   └─ StudyPage.ts
|   └─ SubjectPage.ts
|   └─ InvestigatorPage.ts
|
|   └─ features/
|       └─ login.feature
|       └─ study.feature
|       └─ subject.feature
|       └─ investigator.feature
|
|   └─ stepDefinitions/
|       └─ login.steps.ts
|       └─ study.steps.ts
|       └─ subject.steps.ts
|       └─ hooks.steps.ts
|
|   └─ hooks/
|       └─ hooks.ts
|       └─ retryHooks.ts
|
|   └─ utilities/
|       └─ logger.ts
|       └─ waitHelper.ts
|       └─ screenshotUtil.ts
|       └─ browserFactory.ts
|       └─ encryptionUtil.ts
|       └─ commonActions.ts
|
|   └─ constants/
|       └─ urls.ts
|       └─ messages.ts
|       └─ frameworkConstants.ts
|
|   └─ security/
|       └─ mfaHandler.ts
|       └─ tokenManager.ts
|       └─ sessionManager.ts
|
|   └─ testData/
|       └─ loginData.json
|       └─ studyData.json
|       └─ subjectData.json
|
|   └─ reports/
|       └─ customReporter.ts
```

```
|
|─ allure-results/
|─ allure-report/
|─ screenshots/
|─ videos/
|─ traces/
|─ logs/
|
|─ package.json
|─ tsconfig.json
|─ README.md
└─ azure-pipelines.yml
```

3. package.json

```
{
  "name": "healthcare-ctms-automation",
  "version": "1.0.0",
  "scripts": {
    "test": "npx cucumber-js",
    "test:headed": "npx playwright test --headed",
    "allure:generate": "allure generate ./allure-results --clean",
    "allure:open": "allure open"
  },
  "devDependencies": {
    "@cucumber/cucumber": "latest",
    "@playwright/test": "latest",
    "allure-playwright": "latest",
    "typescript": "latest",
    "ts-node": "latest",
    "dotenv": "latest",
    "winston": "latest"
  }
}
```

4. Playwright Configuration

playwright.config.ts

```
import { PlaywrightTestConfig } from '@playwright/test';

const config: PlaywrightTestConfig = {
  timeout: 60000,
  retries: 1,
  use: {
    headless: true,
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'retain-on-failure'
  },
  projects: [
    {
      name: 'Chromium',
      use: { browserName: 'chromium' }
    },
    {
      name: 'Firefox',
      use: { browserName: 'firefox' }
    },
    {
      name: 'Webkit',
      use: { browserName: 'webkit' }
    }
  ]
};

export default config;
```

5. Base Page Design

BasePage.ts

```
import { Page } from '@playwright/test';

export class BasePage {

  protected page: Page;
```

```
constructor(page: Page) {
    this.page = page;
}

async click(locator: string) {
    await this.page.locator(locator).click();
}

async enterText(locator: string, value: string) {
    await this.page.locator(locator).fill(value);
}

async getText(locator: string) {
    return await this.page.locator(locator).textContent();
}
}
```

6. Login Page

LoginPage.ts

```
import { BasePage } from './BasePage';

export class LoginPage extends BasePage {

    private usernameTextbox = '#username';
    private passwordTextbox = '#password';
    private loginButton = '#loginBtn';

    async launchApplication(url: string) {
        await this.page.goto(url);
    }

    async login(username: string, password: string) {
        await this.enterText(this.usernameTextbox, username);
        await this.enterText(this.passwordTextbox, password);
        await this.click(this.loginButton);
    }
}
```

7. BDD Feature File

login.feature

```
Feature: CTMS Login

Scenario: Successful login with valid credentials
  Given User launches CTMS application
  When User enters valid username and password
  Then User should navigate to dashboard
```

8. Step Definitions

login.steps.ts

```
import { Given, When, Then } from '@cucumber/cucumber';
import { chromium } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

let browser;
let page;
let loginPage;

Given('User launches CTMS application', async function () {
  browser = await chromium.launch();
  page = await browser.newPage();

  loginPage = new LoginPage(page);

  await loginPage.launchApplication('https://ctms-qa.com');
});

When('User enters valid username and password', async function () {
  await loginPage.login('clinical.user', 'Password123');
});

Then('User should navigate to dashboard', async function () {
  await page.waitForURL('**/dashboard');
  await browser.close();
});
```

9. Hooks Implementation

hooks.ts

```
import { Before, After } from '@cucumber/cucumber';

Before(async function () {
  console.log('Execution Started');
});

After(async function () {
  console.log('Execution Completed');
});
```

10. Logging Utility

logger.ts

```
import winston from 'winston';

export const logger = winston.createLogger({
  level: 'info',
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'logs/framework.log' })
  ]
});
```

11. Screenshot Utility

screenshotUtil.ts

```
export async function captureScreenshot(page, fileName: string) {
  await page.screenshot({
    path: `screenshots/${fileName}.png`,
    fullPage: true
  });
}
```

```
    });  
  }  
}
```

12. MFA Authentication Utility

mfaHandler.ts

```
export class MFAHandler {  
  
  async enterOTP(page, otp: string) {  
    await page.fill('#otpTextbox', otp);  
    await page.click('#verifyBtn');  
  }  
}
```

13. JSON Test Data

loginData.json

```
{  
  "validUser": {  
    "username": "clinical.user",  
    "password": "Password123"  
  },  
  "invalidUser": {  
    "username": "invalid.user",  
    "password": "WrongPassword"  
  }  
}
```

14. Allure Reporting

Generate Reports

```
allure generate ./allure-results --clean
allure open
```

Report Features

- Pass/Fail summary
 - Execution duration
 - Failure screenshots
 - Browser execution report
 - Historical trends
-

15. Azure DevOps CI/CD Pipeline

azure-pipelines.yml

```
trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- task: NodeTool@0
  inputs:
    versionSpec: '18.x'

- script: npm install
  displayName: Install Dependencies

- script: npx playwright install
  displayName: Install Playwright Browsers

- script: npm test
  displayName: Execute Automation Tests

- script: allure generate ./allure-results --clean
  displayName: Generate Allure Report
```

16. Healthcare Compliance Automation

HIPAA Compliance Validation

Framework validates:

- Secure login
- Session timeout
- Audit trail
- Encrypted data handling
- Role-based access control

FDA 21 CFR Part 11 Validation

Automation supports:

- Electronic signature validation
- Audit logging verification
- User traceability validation

17. Error Handling Strategy

Supported Failure Handling

- Automatic screenshot capture
- Retry failed tests
- Console log collection
- Trace recording
- Video capture

18. Reusability Strategy

Framework Reusability

Reusable modules include:

- Login workflows
- Navigation components
- Authentication utilities
- Common actions

- Reporting utilities
-

19. Coding Standards

Standards

- No hardcoded waits
 - Centralized locators
 - Generic reusable methods
 - Proper exception handling
 - Strict TypeScript typing
-

20. Recommended Test Coverage

Functional Coverage

- Login
- Patient enrollment
- Clinical study creation
- Subject management
- Investigator workflows
- Trial documentation

Negative Coverage

- Invalid login
 - Session timeout
 - Unauthorized access
 - Invalid study data
-

21. Future Enhancements

Planned Enhancements

- AI self-healing locators
- API automation integration
- Database validation
- Performance testing integration
- BrowserStack cloud execution
- Visual testing support

22. Framework Execution Commands

Execute All Tests

```
npm test
```

Execute Specific Feature

```
npx cucumber-js src/features/login.feature
```

Execute Headed Browser

```
npx playwright test --headed
```

23. Framework Benefits

Benefit	Description
Scalable	Enterprise-grade architecture
Maintainable	Modular reusable design
Secure	Healthcare-compliant automation
Reusable	Shared utilities and components
CI/CD Ready	Azure DevOps integrated
Business Friendly	BDD readable scenarios

24. Conclusion

The Healthcare CTMS Automation Framework provides a scalable, secure, reusable, and enterprise-ready automation solution aligned with Healthcare compliance standards. The framework enables faster regression execution, high maintainability, robust reporting, and reliable cross-browser automation for Healthcare clinical applications.